Congestion-Aware Distributed Service Migration in Cloud-Native Networks

Yu Jiang, Wei Feng, *Senior Member, IEEE,* Ning Ge, *Member, IEEE*

***Abstract*—Service Migration, consider link cost resulted from the migrations of service instances and back-haul communication between user and its agent service instance.**

***Index Terms*—Service Migration, Problem Decomposition, Markov Decision Process, Congestion Control**

I. Introduction

Describe: Resource Allocation (RA), Compare Node RA and Link RA

Contributions:

- An online optimization model is proposed, where we formulated an MINLP problem for joint service migration and resource allocation in cloud-native networks, considering resource constraints on both ANs and backhaul links.
- A two-layer decomposition architecture is proposed to reduce problem complexity and enable distributed optimization. In particular, a virtual subnet partitioning method is introduced to decompose link resource constraints by each AN. To our knowledge, this subnet partitioning based approach is first applied to service migration problems.
- A congestion-aware iterative mechanism is designed, where the link weights are dynamically updated to guide service migration decision, data rate allocation, and path selection. These three aspects are jointly optimized to to alleviate link congestion in the backhaul network.
- Extensive experiments validated the trade-off between overall delay and link overhead. Compared with baseline methods, our approach makes more efficient use of the backhaul link resource and achieves lower overall delay, particularly in network topologies with low connectivity.

II. Related Work

III. Problem Formulation

A. System Model

Fig.1 shows the system architecture for service migration in a bandwidth-constrained backhaul network. First, the backhaul network is modeled as an undirected graph $G(V, E)$, where the node set contains several switches and V Access Nodes (ANs), each equipped with a MEC server and covering a fixed region. For each backhaul link $e \in E$, the communication resource capacity is given by its maximum supported data rate $\overline{R}_e$. For each AN $v \in V$, the computational and bandwidth capacities are denoted as $\overline{C}_v$ and $\overline{W}_v$, respectively. Let N denote the set of mobile users, and u_n denotes the user location. Each user is associated with a personalized agent, which is implemented

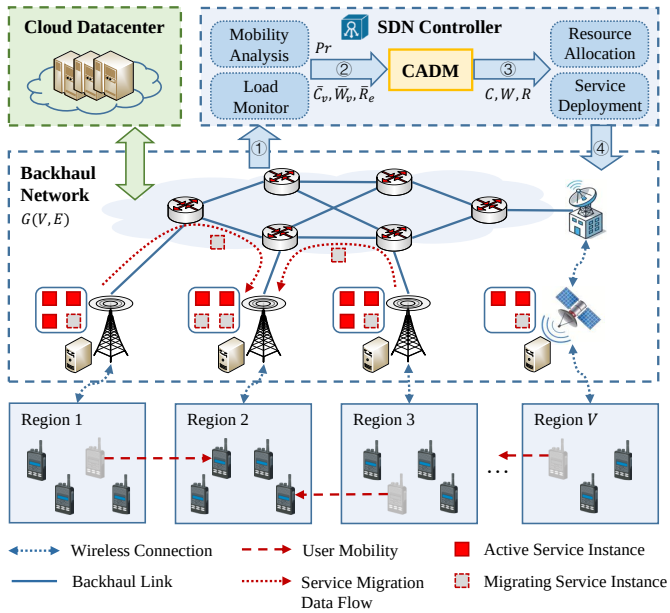

Fig. 1. An illustration of the system model, with service migrations through a bandwidth-constrained backhaul network.

as a service instance hosted on an AN. Thus, the number of users equals that of service instances. A user accesses the AN covering its current region and offloads computing tasks to the AN hosting its service instance. If these two ANs differ, the user request is forwarded via backhaul network G. Then the service instance processes the request and returns the result.

The proposed system runs in discrete time slots, denoted as $t = 1, 2, \ldots, T$. During each time slot t, a user may move into the coverage region of another AN, potentially necessitating the migration of its service instance. For each user $n \in N$, its agent service is characterized by a three-tuple $SV_n = (\phi_n^t, D_n^t, I_n^t)$, where ϕ_n^t denotes the required computational task size (in CPU cycles) during time slot t, D_n^t represents the remote communication data size between the user and its service instance when they are not co-located at the same AN, and I_n^t is the data size to be transferred through G if SV_n is migrated.

B. Optimization Formulation

The set of optimization variables is denoted as (C, W, R), where C and W correspond to node RA, representing the allocation of computing and communication resources on each AN; and R represents both the migration destination and the allocated data rate for transmitting I_n^t, corresponding to service

migration and link RA, respectively. Detailed definitions are listed as follows.

Variables

- C_n^t: continuous scalar, computing resource allocated to user n at time slot t.
- W_n^t: continuous scalar, bandwidth resource allocated to user n at time slot t.
- $R_n^t \in R^{V\times 1}$: continuous vector, $R_n^t(v) \neq 0$ denotes that SV_n deploys on AN v at time t, and the data rate for user n's backhaul communications is $\sum_{v\in V} R_n^t(v)$.

Given R_n^t, we can determine the deployment location of user n's service instance at time t, denoted as $x_n^t = h(R_n^t)$, where function $h(x)$ means finding the position of non-zero element in vector x.

Objective

The optimization objective is defined as the weighted sum of the total delay for each user $n \in N$, denoted as $\xi_n T_n$, where the weight ξ_n is taken as the priority of user n. The total delay consists of three components: computation, communication, and migration delays, as detailed below.

$$\min_{C,W,R} \quad \sum_{n\in N} \xi_n \cdot (T_n^s + T_n^c + T_n^m) \tag{1}$$

First, the computation delay of user n is given by the ratio of its computational task size to the allocated computing resource, denoted as follows:

$$T_n^s = \sum_{t=1}^{T} \frac{\phi_n^t}{C_n^t} \tag{2}$$

Second, the communication delay of user n includes the local access delay T_n^a and the backhaul delay T_n^b:

$$T_n^c = \sum_{t=1}^{T} \left(T_n^a(t) + T_n^b(t)\right) \tag{3}$$

$$T_n^a(t) = \frac{m_n \cdot D_n^t}{W_n^t \cdot \rho(u_n^t)} \tag{3a}$$

$$T_n^b(t) = I_{\{x_n^t \neq u_n^t\}} \cdot \frac{m \cdot D_n^t}{\sum_{v\in V} R_n^t(v)} \tag{3b}$$

where m_n denotes the number of communication rounds between user n and SV_n per time slot, $\rho(u_n^t) = log_2(1 + SNR(u_n^t))$ represents the channel condition at the user's location, and $I\{a\}$ is an indicator function that equals 1 if the condition a is true and 0 otherwise. (3b) means that backhaul communication take place only when $x_n^t \neq u_n^t$.

Third, the migration delay T_n^m occurs only when SV_n is migrated during the current time slot (i.e., $x_n^t \neq x_n^{t-1}$), and is given by:

$$T_n^m = \sum_{t=1}^{T} \left[I_{\{x_n^t \neq x_n^{t-1}\}} \cdot \left(t_n^d + \frac{I_n^t}{\sum\limits_{v\in V} R_n^t(v)} \right) \right] \tag{4}$$

where the constant t_n^d denotes the service downtime, including the shutdown time at the source AN x_n^{t-1} and the startup time at the destination AN x_n^t.

Constraints

The constraints are listed as below.

$$C_n^l \leq C_n^t \leq C_n^u \qquad \forall n\in N, \forall t\in T \tag{5}$$

$$W_n^l \leq W_n^t \leq W_n^u \qquad \forall n\in N, \forall t\in T \tag{6}$$

$$R_n^t(v) \in \left[R_n^l, R_n^u\right] \cup \{0\} \qquad \forall v\in V, \forall n\in N, \forall t\in T \tag{7}$$

$$\sum_{v\in V} I_{\{R_n^t(v)>0\}} = 1 \qquad \forall n\in N, \forall t\in T \tag{8}$$

$$\sum_{n\in N} I_{\{x_n^t=v\}} C_n^t \leq \overline{C}_v \qquad \forall v\in V, \forall t\in T \tag{9}$$

$$\sum_{n\in N} I_{\{x_n^t=v\}} W_n^t \leq \overline{W}_v \qquad \forall v\in V, \forall t\in T \tag{10}$$

$$L_e^t \leq \overline{R}_e \qquad \forall e\in E, \forall t\in T \tag{11}$$

Constraints (5)-(7) specify the feasible ranges of C, W, R, respectively. (8) ensures the uniqueness of the service migration destination. (9) and (10) correspond to node RA, ensuring that the total computing and bandwidth resources occupied on each AN do not exceed its capacity. (11) corresponds to link RA, requiring that the aggregate load L_e^t on each backhaul link $e \in E$ does not exceed its capacity $\overline{R}_e$. Specifically, to derive L_e^t, the concepts of Source-Destination-Pair (SDP) i and routing matrix P are introduced. First, we define a mapping $i = \Pi(v_1, v_2)$ that identifies the SDP with source v_1 and destination v_2, and let I_G denote the set of all SDPs in network $G(V, E)$, where obviously $|I_G| = V^2$. Second, to characterize the routing rules, we introduce a binary matrix $P \in \{0,1\}^{V^2\times E}$, where $P(i, e) = 1$ only when link e lies on the path designated for SDP i. With the above definitions, L_e^t can be derived as follows:

$$L_e^t = \sum_{n\in N} \sum_{v\in V} R_n^t(v) P(i_n^t, e), \quad i_n^t = \Pi(x_n^{t-1}, x_n^t) \tag{12}$$

Based on the definitions and derivations above, the original problem $\mathbf{P_1}(\mathbf{C}, \mathbf{W}, \mathbf{R})$ is summarized as below.

$$\begin{aligned} \min_{C,W,R} \quad & \sum_{n\in N} \xi_n \cdot (T_n^s + T_n^c + T_n^m) \\ s.t. \quad & constraints\ (5) - (11) \end{aligned} \tag{13}$$

As an MINLP problem, $\mathbf{P_1}$ can be reformulated as a classic knapsack problem, which has been proven as NP-hard [xx]. The introduction of the migration variable R significantly increases the problem complexity. If an MDP model is directly constructed based on $\mathbf{P_1}$ to solve the service migration problem for all users $n\in N$, the resulting state space would be extremely high-dimensional. For instance, defining the state at time slot t as $S^t = (x_1^t, ..., x_N^t; u_1^t, ..., u_N^t)$ yields $|S^t| = V^{2N}$. The exponentially growing state space leads to the curse of dimensionality, making it infeasible for traditional policy iteration or Q-learning methods to solve $\mathbf{P_1}$ in practical-scale scenarios. Moreover, as discussed in Section II, applying DRL methods (e.g., DQN, DDPG, PPO) to solve $\mathbf{P_1}$ also faces severe challenges, including inefficient exploration, unstable training, and frequent constraint violations during learning. Therefore, a series of decompositions is required to transform $\mathbf{P_1}$ into low-complexity and fine-grained subproblems, which can be cast as MDPs with smaller and feasible search spaces.

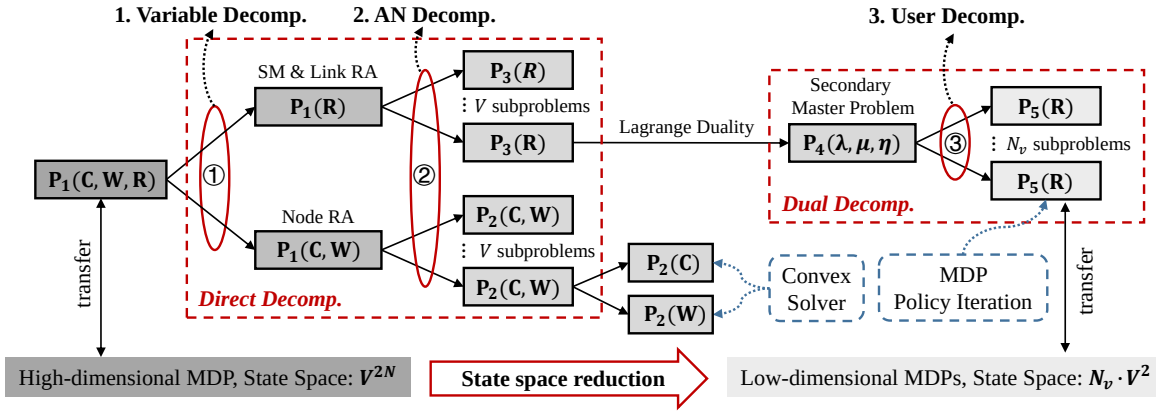

Fig. 2. Detailed two-layer decomposition framework, including the direct and dual decompositions. $\mathbf{P_2(C)}$ and $\mathbf{P_2(W)}$ are convex optimization problems and can be solved by off-the-shelf convex solvers, while $\mathbf{P_5(R)}$ is transformed into an MDP and solved by policy iteration. Compared to the original problem $\mathbf{P_1(C,W,R)}$, the MDP problem transferred from $\mathbf{P_5(R)}$ exhibits a substantially smaller state space (from V^{2N} to $N_v \cdot V^2$).

IV. Problem Decomposition

As illustrated in Fig.2, a two-layer framework is proposed in this section, which consists of two types of decomposition, as shown in the red dashed boxes.

- Direct decomposition: First, variable decomposition is applied to $\mathbf{P_1}$, decoupling node RA from service migration, yielding $\mathbf{P_1(R)}$ (for service migration & link RA) and $\mathbf{P_1(C,W)}$ (for node RA). For each AN, we perform AN decomposition to obtain two sets of subproblems: the node RA subproblems $\mathbf{P_2}$ and the service migration & link RA subproblems $\mathbf{P_3}$.
- Dual decomposition: Lagrange duality is applied to $\mathbf{P_3}$ to obtain its dual problem $\mathbf{P_4}$, which is then further decomposed per user, resulting in the finest-grained subproblem $\mathbf{P_5}$, i.e., the service migration & link RA subproblem for each user.

A detailed elaboration is given below.

A. Direct Decomposition

First, $\mathbf{P_1(C,W,R)}$ is transformed into: (a) $\mathbf{P_1(C,W)}$ with R fixed, corresponding to node RA; (b) $\mathbf{P_1(R)}$ with C, W fixed, corresponding to service migration and link RA. The objective and constraints of these two subproblems are identical to those of (13), and are therefore omitted. In $\mathbf{P_1(C,W)}$, since R is fixed, the objectives and constraints of node RA under each AN are mutually independent. Therefore, it can be decomposed for each AN $v \in V$, producing a total of V subproblems, as detailed below.

$$\mathbf{P_2(C,W)}: \min_{C,W} \quad \sum_{n \in N_v} \xi_n \cdot (T_n^s + T_n^c + T_n^m) \tag{14}$$

$$s.t. \quad \begin{cases} C_n^l \le C_n^t \le C_n^u & \forall n \in N, \forall t \in T \\ W_n^l \le W_n^t \le W_n^u & \forall n \in N, \forall t \in T \\ \sum_{n \in N_v} C_n^t \le \overline{C}_v & \forall t \in T \\ \sum_{n \in N_v} W_n^t \le \overline{W}_v & \forall t \in T \end{cases} \tag{15}$$

Here we define the user subset $N_v = \{n \in N | x_n^t = v\}$, meaning that the whole user set N is partitioned according to current migration destinations $x_n^t = h(R_n^t), \forall n \in N$.

Furthermore, as the allocation of computation (variable C) and communication (variable W) resources are mutually independent in each AN, $\mathbf{P_2(C,W)}$ can be further decomposed into $\mathbf{P_2(C)}$ and $\mathbf{P_2(W)}$ as follows.

$$\mathbf{P_2(C)}: \min_{C} \quad \sum_{n \in N_v} \xi_n \cdot T_n^s \tag{16}$$

$$s.t. \quad \begin{cases} C_n^l \le C_n^t \le C_n^u & \forall n \in N, \forall t \in T \\ \sum_{n \in N_v} C_n^t \le \overline{C}_v & \forall t \in T \end{cases} \tag{17}$$

$$\mathbf{P_2(W)}: \min_{W} \quad \sum_{n \in N_v} \xi_n \cdot T_n^a \tag{18}$$

$$s.t. \quad \begin{cases} W_n^l \le W_n^t \le W_n^u & \forall n \in N, \forall t \in T \\ \sum_{n \in N_v} W_n^t \le \overline{W}_v & \forall t \in T \end{cases} \tag{19}$$

It can be proven that $\mathbf{P_2(C,W)}$ is a convex optimization problem, which can thus be solved directly using off-the-shelf convex optimization solvers (e.g., CPLEX, Gurobi). In particular, for each user $n \in N_v$, if the feasible range constraints in (17) and (19) are relaxed to $C_n^l = 0^+$, $C_n^u = \overline{C}_v$ and $W_n^l = 0^+$, $W_n^u = \overline{W}_v$, the optimal solutions for C and W can be derived in closed form, which are proportional to the product of user weight ξ_n and its resource demand ϕ_n^t or ψ_n^t, as shown below.

$$C_n^{t*} = \frac{\sqrt{\xi_n \phi_n^t} \cdot \overline{C}_v}{\sum_{n \in N_v} \sqrt{\xi_n \phi_n^t}} \tag{20}$$

$$W_n^{t*} = \frac{\sqrt{\xi_n \psi_n^t} \cdot \overline{W}_v}{\sum_{n \in N_v} \sqrt{\xi_n \psi_n^t}}, \quad \psi_n^t = \frac{m D_n^t}{\rho(u_n^t)} \tag{21}$$

B. Virtual Subnet Partition

Different from $\mathbf{P_1(C,W)}$, AN decomposition cannot be directly applied to $\mathbf{P_1(R)}$ due to the presence of link resource constraints (11). For each link $e \in E$, its load L_e^t may originate from any migrating user service across network G, whose migration destinations may vary. Therefore, without knowledge of the migration variable R, constraints (11) cannot be directly decomposed by AN.

To address this issue, the concept of virtual subnet is introduced, where we assign a virtual subnet G_v to each user subset N_v, dedicated to transmitting service migration and backhaul communication data for users $n \in N_v$. Furthermore, for each backhaul link $e \in E$, its resource need to be partitioned based on the loads from each user subset N_v. However, since the exact migration decision is unknown at this stage, it is necessary to estimate these loads, such estimation is detailed in Section V-B. After virtual subnet partitioning, the resource capacity of each backhaul link e is divided into V parts:

$$\overline{R}_e = \sum_{v \in V} \overline{SR}_{e,v}^t \tag{22}$$

where $\overline{SR}_{e,v}^t$ denotes the subnet resource capacity assigned to G_v at time slot t. Consequently, the link resource constraints in (11) is modified as follows.

$$L_{e,v}^t \leq \overline{SR}_{e,v}^t \quad \forall v \in V, \forall e \in E, \forall t \in T \tag{23}$$

$$L_{e,v}^t = \sum_{n \in N_v} R_n^t(v) P(i_n^t, e), \; i_n^t = \Pi(x_n^{t-1}, x_n^t) \tag{24}$$

where $L_{e,v}^t$ denotes the load on link e from user subset N_v at time t. Compared with (11), the number of link resource constraints increases from E to $E \times V$ for each time slot, enabling per-AN decomposition of $\mathbf{P_1}(\mathbf{R})$. The subproblem on each AN v is summarized as follows.

$$\mathbf{P_3}(\mathbf{R}): \min_{R} \quad \sum_{n \in N_v} \xi_n \cdot (T_n^s + T_n^c + T_n^m) \tag{25}$$

$$s.t. \quad \begin{cases} R_n^t(v) \in \left[R_n^l, R_n^u\right] \cup \{0\} & \forall n \in N, \forall t \in T \\ \sum_{v \in V} I_{\{R_n^t(v)>0\}} = 1 & \forall n \in N, \forall t \in T \\ \sum_{n \in N_v} C_n^t \leq \overline{C}_v & \forall t \in T \\ \sum_{n \in N_v} W_n^t \leq \overline{W}_v & \forall t \in T \\ L_{e,v}^t \leq \overline{SR}_{e,v}^t & \forall e \in E, \forall t \in T \end{cases} \tag{26}$$

The formulation of $\mathbf{P_2}(\mathbf{C}, \mathbf{W})$ and $\mathbf{P_3}(\mathbf{R})$ belongs to direct decomposition, which not only reduces complexity but also enables parallel solving for each AN. Next, dual decomposition is applied to $\mathbf{P_3}(\mathbf{R})$ for further complexity reduction.

C. Dual Decomposition

The core idea of dual decomposition is to incorporate coupling constraints into the objective function via Lagrange multipliers (i.e., prices for heterogeneous resources on each node and link), thereby decomposing the original problem into multiple independent subproblems and coordinating these prices to achieve global optimality [xx]. First, the Lagrange function of $\mathbf{P_3}$'s objective (25) is derived as follows.

$$\begin{aligned} L(R, \lambda, \mu, \vec{\eta}) &= \sum_{n \in N_v} \xi_n \cdot (T_n^s + T_n^c + T_n^m) \\ &+ \sum_{t=1}^{T} \lambda^t \cdot \left(\sum_{n \in N_v} C_n^t - \overline{C}_v \right) + \sum_{t=1}^{T} \mu^t \cdot \left(\sum_{n \in N_v} W_n^t - \overline{W}_v \right) \\ &+ \sum_{t=1}^{T} \sum_{e \in E} \eta_e^t \cdot \left(\sum_{n \in N_v} R_n^t(v) P(i_n^t, e) - \overline{R}_e^t(v) \right) \end{aligned} \tag{27}$$

where $i_n^t = \Pi(x_n^{t-1}, x_n^t)$, and $\lambda^t, \mu^t, \vec{\eta}$ are Lagrange multipliers. λ^t, μ^t represent the unit prices of computational and bandwidth resources on AN v, respectively, while $\vec{\eta} = \{\eta_e^t \,|\, e \in E\}$ denotes the unit resource price on each link e for users $n \in N_v$. Then, following transformation is applied to (27) by moving the summation $\sum_{n \in N_v}$ forward.

$$\begin{aligned} L(R, \lambda, \mu, \vec{\eta}) &= \sum_{n \in N_v} \Bigg[\xi_n (T_n^s + T_n^c + T_n^m) + \sum_{t=1}^{T} \lambda^t \cdot C_n^t \\ &+ \sum_{t=1}^{T} \mu^t \cdot W_n^t + \sum_{t=1}^{T} \sum_{e \in E} \eta_e^t \cdot R_n^t(v) P(i_n^t, e) \Bigg] \\ &- \left[\sum_{t=1}^{T} \left(\lambda^t \cdot \overline{C}_v + \mu^t \cdot \overline{W}_v + \sum_{e \in E} \eta_e^t \cdot \overline{R}_e^t(v) \right) \right] \\ &= \sum_{n \in N_v} g_n(R, \lambda, \mu, \vec{\eta}) \; - \; K(\lambda, \mu, \vec{\eta}) \end{aligned} \tag{28}$$

$$\begin{aligned} g_n(R, \lambda, \mu, \vec{\eta}) &= \xi_n (T_n^s + T_n^c + T_n^m) + \sum_{t=1}^{T} \lambda^t \cdot C_n^t \\ &+ \sum_{t=1}^{T} \mu^t \cdot W_n^t + \sum_{t=1}^{T} \sum_{e \in E} \eta_e^t \cdot R_n^t(v) P(i_n^t, e) \end{aligned} \tag{29}$$

$$K(\lambda, \mu, \vec{\eta}) = \sum_{t=1}^{T} \left(\lambda^t \cdot \overline{C}_v + \mu^t \cdot \overline{W}_v + \sum_{e \in E} \eta_e^t \cdot \overline{R}_e^t(v) \right) \tag{30}$$

With the above transformation, the dual problem $\mathbf{P_4}(\mathbf{R})$ is obtained as follows, serving as the secondary master problem.

$$\mathbf{P_4}(\mathbf{R}): \max_{\lambda, \mu, \vec{\eta}} \quad \sum_{n \in N_v} g_n(\lambda, \mu, \vec{\eta}) \; - \; K(\lambda, \mu, \vec{\eta}) \tag{31}$$

$$s.t. \quad \lambda \succeq 0, \; \mu \succeq 0, \vec{\eta} \succeq 0 \tag{32}$$

Subsequently, user decomposition is applied to $\mathbf{P_4}(\mathbf{R})$, yielding a set of the finest-grained subproblems for per-user service migration, denoted as $\mathbf{P_5}(\mathbf{R})$. Note that $K(\lambda, \mu, \vec{\eta})$ (30) is independent of variable R, the optimization objective reduces to only $g_n(R, \lambda, \mu, \vec{\eta})$ (29).

$$\mathbf{P_5}(\mathbf{R}): \min_{R} \quad g_n(R, \lambda, \mu, \vec{\eta}) \tag{33}$$

$$s.t. \quad \begin{cases} R_n^t(v) \in \left[R_n^l, R_n^u\right] \cup \{0\} & \forall t \in T \\ \sum_{v \in V} I_{\{R_n^t(v)>0\}} = 1 & \forall t \in T \end{cases} \tag{34}$$

$\mathbf{P_5}$ focus on the service migration of a single user n, and g_n consists of two parts: the original objective, i.e., the total delay T_n, and two overheads introduced by Lagrangian duality, namely the Node Overhead NO_n and Link Overhead LO_n. Therefore, g_n (33) can be rewritten as follows.

$$g_n = \xi_n T_n + NO_n + LO_n \tag{35}$$

$$NO_n = \sum_{t=1}^{T} \lambda^t \cdot C_n^t + \sum_{t=1}^{T} \mu^t \cdot W_n^t \tag{36}$$

$$LO_n = \sum_{t=1}^{T} \sum_{e \in E} \eta_e^t \cdot R_n^t(v) P(i_n^t, e) \tag{37}$$

Compared with $\mathbf{P_3}$, which only optimizes delay, $\mathbf{P_5}$ incorporates resource overheads into its objective, thereby internalizing resource capacity constraints via price signals. This enables each user to proactively balance its QoS (i.e., delay in this work) against network resource consumption when making service migration decisions. To meet practical requirements, adjustable weight parameters will be introduced in subsequent algorithm design, allowing for flexible control over the relative importance of node and link overheads.

Based on direct and dual decompositions, a two-layer hierarchical framework is developed. As shown in Fig.2, the upper layer serves as the resource coordinator, responsible for node RA and updating the prices of heterogeneous resources, while the lower layer focuses on service migration and link RA by solving $\mathbf{P_5}$ with given prices. Compared with $\mathbf{P_1}$, the finer granularity of $\mathbf{P_5}$ leads to lower complexity and substantially reduced state space of the corresponding MDP model. With user n's state given by $S_n^t = (x_n^t, u_n^t)$, the state space size $|S_n^t| = V^2$. Considering the parallel solution capability of the subproblems for each AN, our decomposition framework reduces the state space from V^{2N} to $N_v{\cdot}V^2$, where N_v denotes the user amount under AN v.

V. Congestion Aware Distributed Service Migration Algorithm

In our layered decomposition framework, the resource prices, i.e., Lagrange multipliers $\lambda^t, \mu^t, \vec{\eta}$ act as interface signals for cross-layer interaction, forming a master-slave closed loop. Built upon this framework, we propose the Congestion-Aware Distributed Service Migration (CADM) algorithm, which is detailed in Alg.1. Overall, CADM consists of the following three nested loops.

- The inner loop (loop 3, lines 16-24) corresponds to dual decomposition, where the service migration subproblems under a given AN are solved with policy iteration.
- The middle loop (loop 2, lines 9-31) corresponds to direct decomposition, where the per-AN problems are solved in parallel, including node & link RA and SM.
- The outer loop (loop 1, lines 6-34) focus on link congestion alleviation by evaluating current optimization results and adjusting link weights.

The CADM algorithm runs in discrete time slots, with the entire three-loop nested iterative process executed on a per-time-slot basis. With the decompositions in Section IV, the node RA on each AN v is addressed by solving $\mathbf{P_2}(\mathbf{C}, \mathbf{W})$ in parallel, as shown in line 13 of Alg.1. Then, the node prices λ^t, μ^t on each AN is updated following the subgradient method for dual decomposition.

$$\lambda^{t,n_2} = \lambda^{t,n_2-1} + \alpha \cdot (\sum_{n \in N_v} C_n^t - \overline{C}_v)^+ \tag{38}$$

$$\mu^{t,n_2} = \mu^{t,n_2-1} + \alpha \cdot (\sum_{n \in N_v} W_n^t - \overline{W}_v)^+ \tag{39}$$

where α is the updating rate. These updated resource price signals are then passed to the inner loop to guide the MDP-based service migration, which is elaborated in the following subsection.

Algorithm 1: Congestion Aware Distributed service Migration (CADM)

1 **for** $t = 2, ..., T$ **do**
2 initialize parameters: $C, W, R, \lambda, \mu, \gamma$;
3 update routing matrix: $P = ShortPath(G, w_e^t)$;
4 $n_1, n_2, n_3 \leftarrow 0$;
5 // **Loop 1**: Congestion-aware outer loop
6 **while** $n_1 \leq N_1^{max}$ **do**
7 $n_1 \leftarrow n_1 + 1$;
8 // **Loop 2**: middle loop for solving P_1
9 **while** $n_2 \leq N_2^{max}$ **do**
10 $n_2 \leftarrow n_2 + 1$;
11 divide user subsets N_v, $v \in V$;
12 **for** $v \in V$, *in parallel* **do**
13 solve $\mathbf{P_2}(\mathbf{C}, \mathbf{W})$ on AN v ;
14 update λ and μ by (38), (39) ;
15 // **Loop 3**: Inner loop for service migration and link RA on AN v
16 **while** $n_3 \leq N_3^{max}$ **do**
17 $n_3 \leftarrow n_3 + 1$;
18 **for** $n \in N_v$ **do**
19 calculate reward RW by (41), (42) ;
20 update $policy = PIter(P_M, RW)$;
21 get action $A_n^{t,n_3} = policy(S_n^t)$;
22 update $\vec{\eta}$ by (45) ;
23 **if** $\Delta g_n^{n_3} < \epsilon_3$ **then**
24 break ;
25 **for** $n \in N_v$ **do**
26 $A_n^{t,n_2} \leftarrow A_n^{t,n_3}$, $R_n^{t,n_2} \leftarrow R_n^{t,n_3}$;
27 estimate policy EP_n^{t,n_2} by (46) ;
28 estimate load $EL_{e,v}^{t,n_2}$ for each v by (47) ;
29 allocate subnet capacities $\overline{SR}_{e,v}^{t,n_2}$ by (48) ;
30 **if** $\Delta C^{n_2} < \epsilon_2$ & $\Delta W^{n_2} < \epsilon_2$ & $\Delta X^{n_2} = 0$ **then**
31 break ;
32 Evaluate current optimization results: $[CG, \gamma_e, w_e^t, E_{vio}, R] = Eval(C, W, R)$;
33 **if** $\Delta CG^{n_1} < \epsilon_1$ & $E_{vio} = \phi$ **then**
34 break ;
35 **Output**: C, W, R

A. MDP based Service Migration

Within the inner loop, the finest-grained $\mathbf{P_5}(\mathbf{R})$ for each single user n is transferred to an MDP, defined as follows.

(1) State: the state in time slot t is defined as $S_n^t = (u_n^t, x_n^t)$, including user n's location u_n^t, and x_n^t is the AN hosting its service instance.

(2) Action: The optimization variable R is converted into the action $A_n^t = (x_n^t, k_n^t)$, where k_n^t denotes the backhaul communication data rate for user n. Here, the continuous interval $\left[R_n^l, R_n^u\right]$ in (34) is discretized into K levels, forming the feasible data rate set K_n with $k_n^t \in K_n$. Note that actions are performed at the start of each time slot, whereas user mobility, characterized by the transition probability matrix P_M, may occur at any arbitrary moment. The state transitions are depicted in Fig.xx, where the intermediate state is defined as

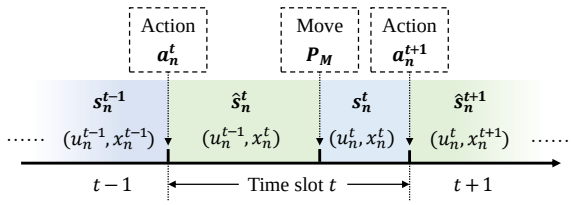

Fig. 3. The state transitions: action a_n^t at the beginning of time slot t causes $s_n^{t-1} \rightarrow \hat{s}_n^t$, and user movement within time slot t causes $\hat{s}_n^t \rightarrow s_n^t$.

$\hat{s}_n^t = (u_n^{t-1}, x_n^t)$.

(3) Reward: the reward function RW is derived from $\mathbf{P_5}$'s objective (33). With the definition of A_n^t, we substitute variables $x = x_n^t$ and $k = k_n^t$ for R in (33), and rewrite g_n as:

$$\begin{aligned} g_n^t(x,k) = &\ \xi_n(T_n^s(t) + T_n^c(t) + T_n^m(t)) \\ &+ \lambda^t C_n^t + \mu^t W_n^t + \sum_{e \in E} \eta_e^t \cdot I_{\{x=v\}} k P(i_n^t, e) \end{aligned} \tag{40}$$

where $n \in N_v$, $t \in T$ and $i_n^t = \Pi(x_n^{t-1}, x)$. Then, the reward of action A_n^t, denoted as $RW(S_n^{t-1}, A_n^t)$, is defined as the reduction in $\mathbf{P_5}$'s objective caused by A_n^t compared with taking no action.

$$RW(S_n^{t-1}, A_n^t) = g_n^t\big|_{\substack{x=x_n^t \\ k=k_n^t}} - g_n^t\big|_{\substack{x=x_n^{t-1} \\ k=k_n^{t-1}}} \tag{41}$$

Subsequently, to make the MDP model congestion-aware and priority-aware, three key parameters $(\omega, \kappa_n, \gamma_e)$ are introduced to modify g_n^t (40). The modified objective is formulated to adapt migration decisions to the link congestion status and user priority differentiation, which is given by:

$$\begin{aligned} g_n^t(x,k,\omega,\kappa_n,\gamma_e) = &\ \xi_n(T_n^s(t) + T_n^c(t) + T_n^m(t)) \\ &+ \omega_N \cdot NO_n^t + \omega_L \cdot LO_n^t \end{aligned} \tag{42}$$

$$NO_n^t = \lambda^t C_n^t + \mu^t W_n^t \tag{43}$$

$$LO_n^t = \kappa_n \sum_{e \in E} \gamma_e \eta_e^t \cdot I_{\{x=v\}} k P(i_n^t, e) \tag{44}$$

First, two weight coefficients ω_N, ω_L are introduced to balance the emphasis on node and link overheads, enabling flexible adjustment of the relative importance of heterogeneous resources. Second, we introduce the user weight $\kappa_n = (1 + \xi_n)^{-1}$ to link overhead LO_n^t, which is negatively correlated with the user priority. This design makes the service migration of low-priority users more constrained by its link overhead. Third, a link-specific weight γ_e is incorporated to revise the link resource price η_e^t, where γ_e is positively correlated with the real-time congestion degree of link e. This induces the MDP model to avoid utilizing already congested links for service migration, thus enabling load-balancing and alleviating congestion on popular links.

With the definitions above, $\mathbf{P_5}$ for each user $n \in N_v$ is converted to a low-complexity MDP and solved with policy iteration, as shown in lines 18-21 of Alg.1. Then, the link prices η_e^t are updated as below, following the same update rule as λ^t and μ^t.

$$\eta_e^{t,n_2} = \eta_e^{t,n_2-1} + \alpha \cdot \Big(\sum_{n \in N_v} R_n^t(v) P(i_n^t, e) - \overline{R}_e^t(v) \Big) \tag{45}$$

Finally, as shown in line 23, the inner loop stops when the change $\Delta g_n^{n_3} = |\sum_{n \in N_v} (g_n^{t,n_3} - g_n^{t,n_3-1})|$ falls below a threshold ϵ_3.

B. Subnet Resource Allocation

After the inner loop converges, the algorithm estimates future policies and allocates resource capacities for each virtual subnets. Corresponding to lines 25-29 of Alg.1, this process is summarized as follows.

$$A_n^t \rightarrow EP_{n,v}^t \rightarrow EL_{e,v}^t \rightarrow \overline{SR}_{e,v}^t$$

$$EP_n^{t,n_2} = \sum_{k=1}^{min\{K,n_2\}} q_p^k \cdot A_n^{t,n_2-k} / \sum_{k=1}^{K} q_p^k \quad n \in N_v \tag{46}$$

$$EL_{e,v}^{t,n_2} = \sum_{n \in N} EP_n^{t,n_2} \cdot R_n^{t,n_2} \cdot P(i_n^t, e) \quad v \in V \tag{47}$$

$$\overline{SR}_{e,v}^{t,n_2} = \frac{(1+\delta)\overline{R}_e \cdot EL_{e,v}^{t,n_2}}{\sum_{v \in V} EL_{e,v}^{t,n_2}} \quad v \in V \tag{48}$$

First, in (46), the estimated policy $EP_{n,v}^t \in [0,1]$ is the probability for service n to migrate to AN v, which is obtained via exponential smoothing over the last K actions, with q_p serving as the attenuation factor. Second, the estimated load on link e originating from user subset N_v, denoted as $EL_{e,v}^{t,n_2}$, is calculated as (47). Based on these estimated loads, $\overline{R}_e$ is partitioned among the virtual subnets, as shown in (48). Note that the inaccuracy in load estimation may lead to resource insufficiency in underestimated subnets. To compensate for such inaccuracy, a positive margin δ is introduced for over-provisioning. However, the over-provisioning may cause the actual load L_e^t on some links to exceed their capacity $\overline{R}_e$. To enforce link resource constraints, a heuristic throttling method is proposed in the outer loop, which is elaborated in Section V-C.

After subnet resource allocation, as shown in line 30, the middle loop stops if the changes in C and W falls below threshold ϵ_2, and the migration decision X remain unchanged.

C. Congestion-Aware Evaluation

Upon convergence of the middle loop for solving the joint problem $\mathbf{P_1}$, as depicted in line 32 of Alg.1, the optimization results (C, W, R) is evaluated based on their influence on link congestion. The evaluation is specified in Function 1. First, for each backhaul link $e \in E$, we calculate its load L_e^t with current optimization results, and evaluate its congestion degree, measured by resource utilization $\theta_e = L_e^t / \overline{R}_e$, which is formulated as the following piecewise function.

$$Con(\theta_e) = \begin{cases} 1, & if \quad 0 \le \theta_e \le \theta_1 \\ 1 + \epsilon_0, & if \quad \theta_1 < \theta_e \le \theta_2 \\ 2 + \epsilon_0, & if \quad \theta_2 < \theta_e \le 1 \end{cases} \tag{49}$$

where $0 < \theta_1 < \theta_2 < 1$ are adjustable thresholds, and ϵ_0 is a positive constant. Then, CADM achieves congestion awareness through two schemes. (1) Path selection: The routing matrix P is updated at the beginning of each time slot by calculating weighted shortest paths for each SDP $i \in I_G$ (line 3 of Alg.1),

Function 1 $Eval(C, W, R)$

1 **// (a) Congestion-aware link weight update**
2 calculate link load L_e^t with C, W, R, then calculate $CG = \sum_{e \in E} Con(\theta_e)$
3 update link weights: $w_e^t = Con(\theta_e)$
4 get congested link set $E_{con} = \{e \in E | \theta_e > \rho_c\}$
5 **for** $e \in E$ **do**
6 **if** $e \in E_{con}$ **then**
7 $\gamma_e^{n_1} = \gamma_e^{n_1 - 1} \cdot [1 + q_c (\theta_e - \rho_c)]$
8 **else**
9 $\gamma_e^{n_1} = \gamma_e^{n_1 - 1} \cdot q_v$
10 **// (b) Heuristic throttling**
11 check link constraints and get violating link set $E_{vio} = \left\{e \in E | L_e^t > \overline{R}_e\right\}$
12 **if** $E_{vio} \neq \phi$ & $n_1 \leq N_1^{max} - 2$ **then**
13 **for** $e \in E_{vio}$ **do**
14 get user set $N_e = \{n \in N | P(i_n^t, e) = 1\}$
15 calculate $\Delta_e = L_e^t - \overline{R}_e$
16 **if** $\Delta_e > 0$ **then**
17 group the users in N_e by ascending priority: $\left[N_e^1, ..., N_e^S\right] = SortP(N_e)$
18 **for** $s = 1 \, to \, S$ **do**
19 sort users in N_e^s by descending data rate: $N_e^s \leftarrow SortR(N_e^s)$
20 **for** $n \in N_e^s$ **do**
21 disable migration of service n: $R_n^t \leftarrow 0$, $\Delta_e \leftarrow \Delta_e - R_n^t$
22 **if** $\Delta_e \leq 0$ **then**
23 return to line 15, check next link in E_{vio}
24 update L_e^t, CG, w_e^t
25

Output: $CG, \gamma_e, w_e^t, E_{vio}, R$

where the congestion degree $Con(\theta_e)$ is taken as link weight w_e^t. (2) Congestion-based pricing: As presented in line 5 of Function 1, the congested link set E_{con} is derived, where ρ_c is the congestion threshold. Links in E_{con} are penalized by raising their coefficient γ_e (line 8), while non-congested links get moderately relaxed (line 10), where q_c and q_v are adjustable parameters within [0,1]. The update of γ_e guides the MDP to avoid overloaded links, forming a distributed congestion pricing mechanism derived from dual decomposition principles, where local migration decisions are aligned with global load balancing across the backhaul network.

The remaining module in Function 1 is heuristic rate throttling. To avoid link resource constraint violations caused by over-provisioning in subnet resource allocation, a lightweight heuristic is adopted as shown in lines 12–25 of Function 1. First, the violating link set E_{vio} is identified. As the outer-loop index n_1 nears the maximum number of iterations, for each $e \in E_{vio}$, we determine its associated user set $N_e = \{n \in N | P(i_n^t, e) = 1\}$, and its required load reduction $\Delta_e = L_e^t - \overline{R}_e$. Then, rate throttling is performed with the *Min Priority & Max Reducible First* principle. Specifically, N_e is grouped by ascending priority (*Min Priority First*). Starting from the lowest-priority group, users in each group are sorted by descending data rate (*Max Reducible First*). Service migration of these users is disabled sequentially until $\Delta_e \leq 0$. This heuristic throttling is capable of eliminating constraint violations while inflicting the least performance degradation on high-priority users.

Finally, as shown in line 33 of Alg.1, the outer loop stops when the change in overall congestion $\Delta CG^{n_1} = CG^{n_1} - CG^{n_1 - 1}$ falls below threshold ϵ_1, enabling control of global network congestion.

VI. Experimental Evaluation

A. Experimental Setup

B. Baseline Approaches

Baselines:

- CADM: Migration + Path Selection + Data Rate
- FixedRate: Migration + Path Selection, with fixed data rate
- FixedPath: Only Migration, with fixed path and data rate
- AlwaysMig: $x_n^t = u_n^t$.

C. Simulation Results

VII. Conclusion and Future Work